

Studentu pazymiu programa
v2.0

Sugeneruota Doxygen 1.17.0

1 Pirmoji užduotis — v1.0 pre-release	1
1.1 Failų struktūra	1
1.1.1 Bendras <code>Data/</code> katalogas	1
1.2 Įdiegimas ir kompiliavimas	1
1.2.1 Reikalavimai	1
1.2.2 Būdas 1 — Makefile (Linux / macOS)	2
1.2.3 Būdas 2 — CMake (Windows / Linux / macOS)	2
1.2.4 Būdas 3 — rankiniu būdu (g++)	2
1.3 Naudojimas	2
1.3.1 Pagrindinė programa	2
1.3.2 Tyrimo programos	2
1.4 Releasai	3
1.4.1 v0.1 — pradinė versija	3
1.4.2 v0.2	3
1.4.3 v0.3	3
1.4.4 v0.4	3
1.4.5 v1.0 -prerelease	3
1.4.6 v1.0	3
1.5 Konteinerių tyrimo rezultatai	3
1.5.1 Nuskaitymas (s)	3
1.5.2 Rūšiavimas (s)	4
1.6 Skaidymo strategijos	4
1.6.1 S1 — du nauji konteineriai (<code>copy_if</code>)	4
1.6.2 Skaidymas (s)	4
1.6.3 S3 — <code>stable_partition + move/splice</code>	4
1.6.4 Skaidymas (s)	4
1.6.5 v1.1	5
2 Hierarchijos Indeksas	7
2.1 Klasių hierarchija	7
3 Klasės Indeksas	9
3.1 Klasės	9
4 Failo Indeksas	11
4.1 Failai	11
5 Klasės Dokumentacija	13
5.1 Studentas Klasė	13
5.1.1 Konstruktoriaus ir Destruktoriaus Dokumentacija	14
5.1.1.1 Studentas() [1/4]	14
5.1.1.2 Studentas() [2/4]	14
5.1.1.3 ~Studentas()	14
5.1.1.4 Studentas() [3/4]	14

5.1.1.5 Studentas() [4/4]	14
5.1.2 Metodų Dokumentacija	14
5.1.2.1 addPazymys()	14
5.1.2.2 apskaiciuoti()	15
5.1.2.3 egz()	15
5.1.2.4 galMed()	15
5.1.2.5 galVid()	15
5.1.2.6 nustatytiEgzIsGalo()	15
5.1.2.7 operator=() [1/2]	15
5.1.2.8 operator=() [2/2]	15
5.1.2.9 paz()	15
5.1.2.10 setEgz()	15
5.1.2.11 skaiciuotiMediana()	15
5.1.2.12 skaiciuotiVidurki()	15
5.1.2.13 spausdinti()	15
5.1.3 Draugiškų Ir Susijusių Funkcijų Dokumentacija	15
5.1.3.1 operator>>	15
5.1.4 Atributų Dokumentacija	16
5.1.4.1 egz_	16
5.1.4.2 gal_med_	16
5.1.4.3 gal_vid_	16
5.1.4.4 paz_	16
5.2 Zmogus Klasė	16
5.2.1 Konstruktoriaus ir Destruktoriaus Dokumentacija	16
5.2.1.1 Zmogus() [1/4]	16
5.2.1.2 Zmogus() [2/4]	17
5.2.1.3 Zmogus() [3/4]	17
5.2.1.4 Zmogus() [4/4]	17
5.2.1.5 ~Zmogus()	17
5.2.2 Metodų Dokumentacija	17
5.2.2.1 operator=() [1/2]	17
5.2.2.2 operator=() [2/2]	17
5.2.2.3 pavarde()	17
5.2.2.4 setPavarde()	17
5.2.2.5 setVardas()	17
5.2.2.6 spausdinti()	17
5.2.2.7 vardas()	17
5.2.3 Atributų Dokumentacija	17
5.2.3.1 pavarde_	17
5.2.3.2 vardas_	18

6 Failo Dokumentacija

19

6.1 funkcijos.cpp Failo Nuoroda	19
6.1.1 Apibrėžimų Dokumentacija	19
6.1.1.1 NOMINMAX	19
6.1.1.2 WIN32_LEAN_AND_MEAN	19
6.1.2 Funkcijos Dokumentacija	20
6.1.2.1 gautiSkaiciu()	20
6.1.2.2 genFaila()	20
6.1.2.3 genPavarde()	20
6.1.2.4 genPazymius()	20
6.1.2.5 genVarda()	20
6.1.2.6 mt()	20
6.1.2.7 skaitytiDeque()	20
6.1.2.8 skaitytiList()	20
6.1.2.9 skaitytiVector()	20
6.1.2.10 skirstytiDeque()	21
6.1.2.11 skirstytiList()	21
6.1.2.12 skirstytiVector()	21
6.2 README.md Failo Nuoroda	21
6.3 struktura.h Failo Nuoroda	21
6.3.1 Apibrėžimų Dokumentacija	22
6.3.1.1 NOMINMAX	22
6.3.1.2 WIN32_LEAN_AND_MEAN	22
6.3.2 Funkcijos Dokumentacija	22
6.3.2.1 gautiSkaiciu()	22
6.3.2.2 genFaila()	22
6.3.2.3 genPavarde()	22
6.3.2.4 genPazymius()	22
6.3.2.5 genVarda()	22
6.3.2.6 operator<<()	23
6.3.2.7 operator>>()	23
6.3.2.8 skaitytiDeque()	23
6.3.2.9 skaitytiIsFailo()	23
6.3.2.10 skaitytiList()	23
6.3.2.11 skaitytiVector()	23
6.3.2.12 skirstytiDeque()	23
6.3.2.13 skirstytiList()	23
6.3.2.14 skirstytiVector()	23
6.3.2.15 spausdintiRezultatus()	24
6.3.2.16 splitStudents()	24
6.3.2.17 test1()	24
6.3.2.18 test2()	24
6.3.2.19 vykdytiVector()	24

6.4 struktura.h	24
6.5 Uzd2.cpp Failo Nuoroda	28
6.5.1 Funkcijos Dokumentacija	28
6.5.1.1 main()	28
6.6 vektorius.cpp Failo Nuoroda	28
6.6.1 Funkcijos Dokumentacija	29
6.6.1.1 skaitytiFailo()	29
6.6.1.2 spausdintiRezultatus()	29
6.6.1.3 splitStudents()	29
6.6.1.4 vykdytiVector()	29
6.6.2 Kintamojo Dokumentacija	29
6.6.2.1 DATA_DIR	29
Rodyklė	31

skyrius 1

Pirmoji užduotis — v1.0 pre-release

C++17 programa studentų galutinių pažymių skaičiavimui, rūšiavimui ir skaidymui į grupes. Versija v1.0 prideda konteinerių greičio tyrimą (`std::vector`, `std::list`, `std::deque`).

1.1 Failų struktūra

```
...
struktura.h      -- bendras antraštės failas (struktūra, using, prototipai)
funkcijos.cpp    -- pagalbinės funkcijos (generatoriai, matematika, I/O)
vektorius.cpp    -- pagrindinio meniu logika (v0.4, std::vector)
testavimas.cpp   -- test1 (failų kūrimas), test2 (vector lyginimas)
Uzd1.cpp         -- pagrindinis įėjimas (projektas Uzd1)
vector.cpp       -- v1.0 tyrimas: std::vector → vykdomasis: stud_Vector
list.cpp         -- v1.0 tyrimas: std::list   → vykdomasis: stud_List
deque.cpp        -- v1.0 tyrimas: std::deque  → vykdomasis: stud_Deque
CMakeLists.txt   -- build konfigūracija (Windows / Linux / macOS)
Makefile         -- build konfigūracija (Linux / macOS, be CMake)
Data/            -- sugeneruoti testavimo failai (sukuriamas automatiškai)
    studentailk.txt
    studentail0k.txt
    studentail00k.txt
    studentailM.txt
    studentail10M.txt
```

1.1.1 Bendras Data/ katalogas

Visos trys tyrimo programos (`vector`, `list`, `deque`) ir senoji (v0.4) programa skaito duomenų failus iš to paties `Data/` katalogo. Tai reiškia, kad failus reikia sugeneruoti tik vieną kartą — bet kuri programa gali tai padaryti, o likusios naudos tuos pačius failus.

Katalogas sukuriamas automatiškai, kai pasirenkate failų generavimą. Jis atsiranda šalia vykdomojo failo.

```
Data/
studentailk.txt      (~136 KB)
studentail0k.txt     (~1.4 MB)
studentail00k.txt    (~14 MB)
studentailM.txt      (~136 MB)
studentail10M.txt    (~1.4 GB)
```

1.2 Įdiegimas ir kompiliavimas

1.2.1 Reikalavimai

- C++17 palaikantis kompiliatorius (g++ 9+ arba clang++ 9+ arba MSVC 2019+)
- CMake 3.14+ (neprivaloma — galima kompiliuoti rankiniu būdu)

1.2.2 Būdas 1 — Makefile (Linux / macOS)

Nereikia jokių papildomų įrankių — tik `g++` ir `make`, kurie dažniausiai jau įdiegti.

1. Patikrink ar įrankiai yra:

```
g++ --version
make --version
```

Jei `g++` nėra — įdiek:

```
# Ubuntu / Debian
sudo apt install build-essential
```

```
# macOS
xcode-select --install
```

2. Eik į projekto katalogą:

```
cd /kelias/iki/projekto
```

Tai katalogas, kuriame yra `Makefile` ir visi `.cpp` failai.

3. Kompiliuok:

```
# Sukompiluoja visus keturis projektus iš karto
make
```

```
# Arba tik vieną konkretų
make Uzd1
make stud_Vector
make stud_List
make stud_Deque
```

4. Paleisk:

```
./Uzd1
./stud_Vector
./stud_List
./stud_Deque
```

5. Ištrinti sukompilijuotus failus:

```
make clean
```

Vykdomieji failai atsiranda tiesiai projekto kataloge šalia `.cpp` failų — `Data/` katalogas randamas teisingai.

1.2.3 Būdas 2 — CMake (Windows / Linux / macOS)

Linux / macOS:

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build
```

Windows su MinGW:

```
cmake -B build -G "MinGW Makefiles" -DCMAKE_BUILD_TYPE=Release
cmake --build build
```

Windows su Visual Studio — atsidaryk `CMakeLists.txt` tiesiai per Visual Studio (File → Open → CMake), pasirink konfigūraciją `Release` ir spausk `Build All`.

Vykdomieji failai rašomi tiesiai į projekto katalogą (ne į `build/`), kad `Data/` aplankas būtų randamas teisingai.

1.2.4 Būdas 3 — rankiniu būdu (g++)

```
g++ -O2 -std=c++17 Uzd1.cpp vektorius.cpp testavimas.cpp funkcijos.cpp -o Uzd1
g++ -O2 -std=c++17 vector.cpp funkcijos.cpp -o stud_Vector
g++ -O2 -std=c++17 list.cpp funkcijos.cpp -o stud_List
g++ -O2 -std=c++17 deque.cpp funkcijos.cpp -o stud_Deque
```

1.3 Naudojimas

1.3.1 Pagrindinė programa

Interaktyvus meniu. Leidžia įvesti studentus ranka, generuoti juos arba nuskaityti iš failo, rūšiuoti ir skaidyti į grupes.

1.3.2 Tyrimo programos

Paleiskite bet kurią iš trijų programų. Pirmą kartą pasirinkite failų generavimą — jie bus sukurti `Data/` kataloge ir visų programų bendrai naudojami.

```
./stud_Vector # sugeneruoja Data/ ir atlieka tyrimą
./stud_List   # naudoja tuos pačius Data/ failus
./stud_Deque  # naudoja tuos pačius Data/ failus
```

Failų generuoti antrą kartą nereikia — tiesiog atsakykite 0 į klausimą apie generavimą.

1.4 Releasai

1.4.1 v0.1 — pradinė versija

Bazinė struktūra: `Studentas` su `std::vector<int>` pažymiams, rankinis įvedimas, galutinio pažymio skaičiavimas vidurkiu ir mediana.

1.4.2 v0.2

Pridėtas skaitymas iš failo ir rezultatų išvedimas į failą.

1.4.3 v0.3

Pridėtas automatinis duomenų generavimas (vardai, pavardės, pažymiai). Pridėta klaidų apdorojimas `gautiSkaiciu` funkcijai.

1.4.4 v0.4

Pridėtas studentų skaidymas į dvi grupes: `kietai` (galutinis ≥ 5.0) ir `tinginiai` (galutinis < 5.0). Pridėtas failų kūrimo ir duomenų apdorojimo greičio tyrimas (`test1`, `test2`). Naudojama `copy_if` į du naujus `std::vector` konteinerius.

1.4.5 v1.0 -prerelease

Pridėtos trys atskiros tyrimo programos (`vector`, `list`, `deque`), matuojančios:

- duomenų nuskaitymą iš failo;
- studentų rūšiavimą mažėjančia galutinio pažymio tvarka;

1.4.6 v1.0

- skaidymą į dvi grupes dviem strategijomis (S1 ir S3).

1.5 Konteinerių tyrimo rezultatai

Testavimo sistema: (ASUS VIVOBOOK S 14)

Parametras	Reikšmė
CPU	AMD Ryzen AI 9
RAM	24 GB
Saugykla	1 TB SSD (NVMe)
OS	Windows 11
Kompiliatorius	g++

Žemiau pateikti rezultatai, gauti testavimo metu (su release):

1.5.1 Nuskaitymas (s)

Failas	vector	list	deque
1 000 įrašų	0.0050	0.0042	0.0050
10 000 įrašų	0.0411	0.0407	0.0406

Failas	vector	list	deque
100 000 įrašų	0.3905	0.3891	0.3811
1 000 000 įrašų	3.7790	3.7911	3.7534
10 000 000 įrašų	38.2739	50.6590	49.0184

Nuskaitymas visur panašus — skirtumai mažesni nei failo I/O triukšmas.

1.5.2 Rūšiavimas (s)

Failas	vector	list	deque
1 000 įrašų	0.0001	0.0000	0.0001
10 000 įrašų	0.0009	0.0007	0.0011
100 000 įrašų	0.0073	0.0193	0.0128
1 000 000 įrašų	0.0855	0.5396	0.2673
10 000 000 įrašų	0.7860	12.2730	4.2492

`std::vector` rūšiuojamas greičiausiai dėl gretimos atminties (CPU talpykla). `list` yra $\sim 2.5\times$ lėtesnis 100k atveju. `deque` artimas vektoriui.

1.6 Skaidymo strategijos

1.6.1 S1 — du nauji konteineriai (`copy_if`)

1.6.2 Skaidymas (s)

Failas	vector	list	deque
1 000 įrašų	0.0002	0.0001	0.0001
10 000 įrašų	0.0016	0.0021	0.0014
100 000 įrašų	0.0215	0.0330	0.0236
1 000 000 įrašų	0.2446	0.3862	0.2574
10 000 000 įrašų	3.1586	5.2494	3.2320

Originalas nekeičiamas. Kiekvienas studentas saugomas **dviejose** vietose atmintyje. Du pilni praėjimai per duomenis.

1.6.3 S3 — `stable_partition + move/splice`

1.6.4 Skaidymas (s)

Failas	vector	list	deque
1 000 įrašų	0.0000	0.0000	0.0000
10 000 įrašų	0.0001	0.0001	0.0003
100 000 įrašų	0.0011	0.0056	0.0045
1 000 000 įrašų	0.0130	0.0736	0.0636

Failas	vector	list	deque
10 000 000 įrašų	0.1820	0.8060	0.7349

Duomenys tvarkomi **vietoje** — nereikia papildomos atminties iteracijoms. Vargšiškai perkeliama (`move`) be kopijavimo. `list` versijoje vietoj `move` naudojamas `splice()` — tik rodyklių pakeitimas.

1.6.5 v1.1

Struct pakeista į Class Su visual studio

Skaidymas naudojant vector (be optimizacijos):

Failas	vector
1 000 įrašų	0.0000
10 000 įrašų	0.0001
100 000 įrašų	0.0011
1 000 000 įrašų	0.0125
10 000 000 įrašų	0.2181

(su /O1)

Failas	vector
1 000 įrašų	0.0000
10 000 įrašų	0.0001
100 000 įrašų	0.0010
1 000 000 įrašų	0.0124
10 000 000 įrašų	0.3042

(su /O2)

Failas	vector
1 000 įrašų	0.0000
10 000 įrašų	0.0001
100 000 įrašų	0.0009
1 000 000 įrašų	0.0133
10 000 000 įrašų	0.1656

Su cmake:

(su /O1)

Failas	vector
1 000 įrašų	0.0000
10 000 įrašų	0.0001
100 000 įrašų	0.0021
1 000 000 įrašų	0.0273
10 000 000 įrašų	0.3471

(su /O2)

Failas	vector
1 000 įrašų	0.0000
10 000 įrašų	0.0003
100 000 įrašų	0.0025
1 000 000 įrašų	0.0282
10 000 000 įrašų	0.3588

(su /O3)

Failas	vector
1 000 įrašų	0.0000
10 000 įrašų	0.0003
100 000 įrašų	0.0023
1 000 000 įrašų	0.0272
10 000 000 įrašų	0.4811

Nors naudojant cmake dalinimas užtrunka ilgiau (apie 2x), tačiau failų nuskaitymas vyks ta kelis kart greičiau. Tokio dydžio duomenimis programa bendrai veikia greičiau. O naudojant Visual Studio ar CMake vis tiek bus greičiau nei naudojant struct be optimizacijos.

skyrius 2

Hierarchijos Indeksas

2.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

Zmogus	16
Studentas	13

skyrius 3

Klasės Indeksas

3.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

Studentas	13
Zmogus	16

skyrius 4

Failo Indeksas

4.1 Failai

Visų failų sąrašas su trumpais aprašymais:

funkcijos.cpp	19
struktura.h	21
Uzd2.cpp	28
vektorius.cpp	28

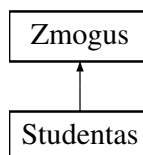
skyrius 5

Klasės Dokumentacija

5.1 Studentas Klasė

```
#include <struktura.h>
```

Paveldimumo diagrama Studentas:



Vieši Metodai

- `Studentas ()`=default
- `Studentas (string v, string p, vector< int > paz, int egz, int metodos)`
- `~Studentas ()` override=default
- `Studentas (const Studentas &kitas)`
- `Studentas & operator= (const Studentas &kitas)`
- `Studentas (Studentas &&kitas)` noexcept
- `Studentas & operator= (Studentas &&kitas)` noexcept
- void `spausdinti (ostream &os)` const override
- const vector< int > & `paz ()` const
- int `egz ()` const
- double `galVid ()` const
- double `galMed ()` const
- void `setEgz (int e)`
- void `addPazymys (int p)`
- void `nustatytiEgzIsGalo ()`
- double `skaiciuotiVidurki ()` const
- double `skaiciuotiMediana ()` const
- void `apskaiciuoti (int metodos)`

Vieši Metodai inherited from Zmogus

- `Zmogus ()`=default
- `Zmogus (string v, string p)`
- `Zmogus (const Zmogus &)=default`
- `Zmogus (Zmogus &&)` noexcept=default
- `Zmogus & operator= (const Zmogus &)=default`
- `Zmogus & operator= (Zmogus &&)` noexcept=default
- virtual `~Zmogus ()`=default

- const string & `vardas` () const
- const string & `pavarde` () const
- void `setVardas` (const string &v)
- void `setPavarde` (const string &p)

Privatūs Atributai

- vector< int > `paz_`
- int `egz_` = 0
- double `gal_vid_` = 0.0
- double `gal_med_` = 0.0

Draugai

- istream & `operator>>` (istream &is, `Studentas` &st)

Additional Inherited Members

Apsaugoti Atributai inherited from `Zmogus`

- string `vardas_`
- string `pavarde_`

5.1.1 Konstruktoriaus ir Destruktoriaus Dokumentacija

5.1.1.1 `Studentas()` [1/4]

```
Studentas::Studentas () [default]
```

5.1.1.2 `Studentas()` [2/4]

```
Studentas::Studentas (
    string v,
    string p,
    vector< int > paz,
    int egz,
    int metodas) [inline]
```

5.1.1.3 `~Studentas()`

```
Studentas::~~Studentas () [override], [default]
```

5.1.1.4 `Studentas()` [3/4]

```
Studentas::Studentas (
    const Studentas & kitas) [inline]
```

5.1.1.5 `Studentas()` [4/4]

```
Studentas::Studentas (
    Studentas && kitas) [inline], [noexcept]
```

5.1.2 Metodų Dokumentacija

5.1.2.1 `addPazymys()`

```
void Studentas::addPazymys (
    int p) [inline]
```

5.1.2.2 apskaiciuoti()

```
void Studentas::apskaiciuoti (
    int metodas) [inline]
```

5.1.2.3 egz()

```
int Studentas::egz () const [inline]
```

5.1.2.4 galMed()

```
double Studentas::galMed () const [inline]
```

5.1.2.5 galVid()

```
double Studentas::galVid () const [inline]
```

5.1.2.6 nustatytiEgzIsGalo()

```
void Studentas::nustatytiEgzIsGalo () [inline]
```

5.1.2.7 operator=() [1/2]

```
Studentas & Studentas::operator= (
    const Studentas & kitas) [inline]
```

5.1.2.8 operator=() [2/2]

```
Studentas & Studentas::operator= (
    Studentas && kitas) [inline], [noexcept]
```

5.1.2.9 paz()

```
const vector< int > & Studentas::paz () const [inline]
```

5.1.2.10 setEgz()

```
void Studentas::setEgz (
    int e) [inline]
```

5.1.2.11 skaiciuotiMediana()

```
double Studentas::skaiciuotiMediana () const [inline]
```

5.1.2.12 skaiciuotiVidurki()

```
double Studentas::skaiciuotiVidurki () const [inline]
```

5.1.2.13 spausdinti()

```
void Studentas::spausdinti (
    ostream & os) const [inline], [override], [virtual]
Realizuoja Zmogus.
```

5.1.3 Draugiškų Ir Susijusių Funkcijų Dokumentacija

5.1.3.1 operator>>

```
istream & operator>> (
    istream & is,
    Studentas & st) [friend]
```

5.1.4 Atributų Dokumentacija

5.1.4.1 egz_

```
int Studentas::egz_ = 0 [private]
```

5.1.4.2 gal_med_

```
double Studentas::gal_med_ = 0.0 [private]
```

5.1.4.3 gal_vid_

```
double Studentas::gal_vid_ = 0.0 [private]
```

5.1.4.4 paz_

```
vector<int> Studentas::paz_ [private]
```

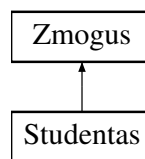
Dokumentacija šiai klasei sugeneruota iš šio failo:

- [struktura.h](#)

5.2 Zmogus Klasė

```
#include <struktura.h>
```

Paveldimumo diagrama Zmogus:



Vieši Metodai

- [Zmogus](#) ()=default
- [Zmogus](#) (string v, string p)
- [Zmogus](#) (const [Zmogus](#) &)=default
- [Zmogus](#) ([Zmogus](#) &&) noexcept=default
- [Zmogus](#) & [operator=](#) (const [Zmogus](#) &)=default
- [Zmogus](#) & [operator=](#) ([Zmogus](#) &&) noexcept=default
- virtual [~Zmogus](#) ()=default
- const string & [vardas](#) () const
- const string & [pavarde](#) () const
- void [setVardas](#) (const string &v)
- void [setPavarde](#) (const string &p)
- virtual void [spausdinti](#) (ostream &os) const =0

Apsaugoti Atributai

- string [vardas_](#)
- string [pavarde_](#)

5.2.1 Konstruktoriaus ir Destruktoriaus Dokumentacija

5.2.1.1 Zmogus() [1/4]

```
Zmogus::Zmogus () [default]
```

5.2.1.2 Zmogus() [2/4]

```
Zmogus::Zmogus (
    string v,
    string p) [inline]
```

5.2.1.3 Zmogus() [3/4]

```
Zmogus::Zmogus (
    const Zmogus & ) [default]
```

5.2.1.4 Zmogus() [4/4]

```
Zmogus::Zmogus (
    Zmogus && ) [default], [noexcept]
```

5.2.1.5 ~Zmogus()

```
virtual Zmogus::~Zmogus () [virtual], [default]
```

5.2.2 Metodų Dokumentacija

5.2.2.1 operator=() [1/2]

```
Zmogus & Zmogus::operator= (
    const Zmogus & ) [default]
```

5.2.2.2 operator=() [2/2]

```
Zmogus & Zmogus::operator= (
    Zmogus && ) [default], [noexcept]
```

5.2.2.3 pavarde()

```
const string & Zmogus::pavarde () const [inline]
```

5.2.2.4 setPavarde()

```
void Zmogus::setPavarde (
    const string & p) [inline]
```

5.2.2.5 setVardas()

```
void Zmogus::setVardas (
    const string & v) [inline]
```

5.2.2.6 spausdinti()

```
virtual void Zmogus::spausdinti (
    ostream & os) const [pure virtual]
```

Realizuota [Studentas](#).

5.2.2.7 vardas()

```
const string & Zmogus::vardas () const [inline]
```

5.2.3 Atributų Dokumentacija

5.2.3.1 pavarde_

```
string Zmogus::pavarde_ [protected]
```

5.2.3.2 vardas_

`string Zmogus::vardas_ [protected]`

Dokumentacija šiai klasei sugeneruota iš šio failo:

- [struktura.h](#)

skyrius 6

Failo Dokumentacija

6.1 funkcijos.cpp Failo Nuoroda

```
#include <windows.h>
#include "struktura.h"
```

Apibrėžimai

- #define `NOMINMAX`
- #define `WIN32_LEAN_AND_MEAN`

Funkcijos

- static std::mt19937 `mt` (std::chrono::steady_clock::now().time_since_epoch().count())
- string `genVarda` ()
- string `genPavarde` (string vardas)
- void `genPazymius` (vector< int > &paz, int &egz)
- void `genFaila` (const string &failas, int kiek)
- int `gautiSkaiciu` (string info, int min, int max)
- void `skaitytiVector` (const string &failas, vector< `Studentas` > &grupe, int metodash)
- void `skaitytiList` (const string &failas, list< `Studentas` > &grupe, int metodash)
- void `skaitytiDeque` (const string &failas, deque< `Studentas` > &grupe, int metodash)
- void `skirstytiVector` (const vector< `Studentas` > &visi, vector< `Studentas` > &kieti, vector< `Studentas` > &tinginiai)
- void `skirstytiList` (const list< `Studentas` > &visi, list< `Studentas` > &kieti, list< `Studentas` > &tinginiai)
- void `skirstytiDeque` (const deque< `Studentas` > &visi, deque< `Studentas` > &kieti, deque< `Studentas` > &tinginiai)

6.1.1 Apibrėžimų Dokumentacija

6.1.1.1 NOMINMAX

```
#define NOMINMAX
```

6.1.1.2 WIN32_LEAN_AND_MEAN

```
#define WIN32_LEAN_AND_MEAN
```

6.1.2 Funkcijos Dokumentacija

6.1.2.1 gautiSkaiciu()

```
int gautiSkaiciu (  
    string info,  
    int min,  
    int max)
```

6.1.2.2 genFaila()

```
void genFaila (  
    const string & failas,  
    int kiek)
```

6.1.2.3 genPavarde()

```
string genPavarde (  
    string vardas)
```

6.1.2.4 genPazymius()

```
void genPazymius (  
    vector< int > & paz,  
    int & egz)
```

6.1.2.5 genVarda()

```
string genVarda ()
```

6.1.2.6 mt()

```
std::mt19937 mt (  
    std::chrono::steady_clock::now().time_since_epoch().count() ) [static]
```

6.1.2.7 skaitytiDeque()

```
void skaitytiDeque (  
    const string & failas,  
    deque< Studentas > & grupe,  
    int metodas)
```

6.1.2.8 skaitytiList()

```
void skaitytiList (  
    const string & failas,  
    list< Studentas > & grupe,  
    int metodas)
```

6.1.2.9 skaitytiVector()

```
void skaitytiVector (  
    const string & failas,  
    vector< Studentas > & grupe,  
    int metodas)
```

6.1.2.10 skirstytiDeque()

```
void skirstytiDeque (
    const deque< Studentas > & visi,
    deque< Studentas > & kieti,
    deque< Studentas > & tinginiai)
```

6.1.2.11 skirstytiList()

```
void skirstytiList (
    const list< Studentas > & visi,
    list< Studentas > & kieti,
    list< Studentas > & tinginiai)
```

6.1.2.12 skirstytiVector()

```
void skirstytiVector (
    const vector< Studentas > & visi,
    vector< Studentas > & kieti,
    vector< Studentas > & tinginiai)
```

6.2 README.md Failo Nuoroda**6.3 struktura.h Failo Nuoroda**

```
#include <iostream>
#include <vector>
#include <string>
#include <iomanip>
#include <algorithm>
#include <random>
#include <chrono>
#include <limits>
#include <fstream>
#include <sstream>
#include <list>
#include <deque>
#include <filesystem>
#include <windows.h>
```

Klasės

- class [Zmogus](#)
- class [Studentas](#)

Apibrėžimai

- #define [NOMINMAX](#)
- #define [WIN32_LEAN_AND_MEAN](#)

Funkcijos

- ostream & [operator<<](#) (ostream &os, const [Zmogus](#) &z)
- istream & [operator>>](#) (istream &is, [Studentas](#) &st)
- int [gautiSkaiciu](#) (string info, int min, int max)
- string [genVarda](#) ()
- string [genPavarde](#) (string vardas)

- void `genPazymius` (vector< int > &paz, int &egz)
- void `genFaila` (const string &failas, int kiek)
- void `skaitytiVector` (const string &failas, vector< `Studentas` > &grupe, int metodas)
- void `skaitytiList` (const string &failas, list< `Studentas` > &grupe, int metodas)
- void `skaitytiDeque` (const string &failas, deque< `Studentas` > &grupe, int metodas)
- void `skirstytiVector` (const vector< `Studentas` > &visi, vector< `Studentas` > &kieti, vector< `Studentas` > &tinginiai)
- void `skirstytiList` (const list< `Studentas` > &visi, list< `Studentas` > &kieti, list< `Studentas` > &tinginiai)
- void `skirstytiDeque` (const deque< `Studentas` > &visi, deque< `Studentas` > &kieti, deque< `Studentas` > &tinginiai)
- void `skaitytiFailo` (const string &failas, vector< `Studentas` > &grupe, int metodas)
- void `spausdintiRezultatus` (const vector< `Studentas` > &grupe, int rodyti, const string &failas)
- void `splitStudents` (const vector< `Studentas` > &visi, vector< `Studentas` > &kieti, vector< `Studentas` > &tinginiai, int metodas)
- void `vykdytiVector` ()
- void `test1` ()
- void `test2` (const string &failas, int metodas)

6.3.1 Apibrėžimų Dokumentacija

6.3.1.1 NOMINMAX

```
#define NOMINMAX
```

6.3.1.2 WIN32_LEAN_AND_MEAN

```
#define WIN32_LEAN_AND_MEAN
```

6.3.2 Funkcijos Dokumentacija

6.3.2.1 gautiSkaiciu()

```
int gautiSkaiciu (
    string info,
    int min,
    int max)
```

6.3.2.2 genFaila()

```
void genFaila (
    const string & failas,
    int kiek)
```

6.3.2.3 genPavarde()

```
string genPavarde (
    string vardas)
```

6.3.2.4 genPazymius()

```
void genPazymius (
    vector< int > & paz,
    int & egz)
```

6.3.2.5 genVarda()

```
string genVarda ()
```

6.3.2.6 operator<<()

```
ostream & operator<< (
    ostream & os,
    const Zmogus & z) [inline]
```

6.3.2.7 operator>>()

```
istream & operator>> (
    istream & is,
    Studentas & st) [inline]
```

6.3.2.8 skaitytiDeque()

```
void skaitytiDeque (
    const string & failas,
    deque< Studentas > & grupe,
    int metodas)
```

6.3.2.9 skaitytiIsFailo()

```
void skaitytiIsFailo (
    const string & failas,
    vector< Studentas > & grupe,
    int metodas)
```

6.3.2.10 skaitytiList()

```
void skaitytiList (
    const string & failas,
    list< Studentas > & grupe,
    int metodas)
```

6.3.2.11 skaitytiVector()

```
void skaitytiVector (
    const string & failas,
    vector< Studentas > & grupe,
    int metodas)
```

6.3.2.12 skirstytiDeque()

```
void skirstytiDeque (
    const deque< Studentas > & visi,
    deque< Studentas > & kieti,
    deque< Studentas > & tinginiai)
```

6.3.2.13 skirstytiList()

```
void skirstytiList (
    const list< Studentas > & visi,
    list< Studentas > & kieti,
    list< Studentas > & tinginiai)
```

6.3.2.14 skirstytiVector()

```
void skirstytiVector (
    const vector< Studentas > & visi,
    vector< Studentas > & kieti,
    vector< Studentas > & tinginiai)
```

6.3.2.15 spausdintiRezultatus()

```
void spausdintiRezultatus (
    const vector< Studentas > & grupe,
    int rodyti,
    const string & failas)
```

6.3.2.16 splitStudents()

```
void splitStudents (
    const vector< Studentas > & visi,
    vector< Studentas > & kieti,
    vector< Studentas > & tinginiai,
    int metodas)
```

6.3.2.17 test1()

```
void test1 ()
```

6.3.2.18 test2()

```
void test2 (
    const string & failas,
    int metodas)
```

6.3.2.19 vykdytiVector()

```
void vykdytiVector ()
```

6.4 struktura.h

Eiti į šio failo dokumentaciją.

```
00001 // struktura.h
00002 #pragma once
00003 // struktura.h -- v1.5
00004 //
00005 // Klasių hierarchija:
00006 // Zmogus (abstrakti bazinė klasė)
00007 // Studentas (paveldėjusi)
00008 //
00009 // NOMINMAX būtina prieš windows.h --
00010 // be jos windows.h apibrėžia min/max kaip makrosus ir sugadina
00011 // std::numeric_limits<T>::max() bei std::min/std::max kvietimus.
00012 #define NOMINMAX
00013 #define WIN32_LEAN_AND_MEAN
00014
00015 #include <iostream>
00016 #include <vector>
00017 #include <string>
00018 #include <iomanip>
00019 #include <algorithm>
00020 #include <random>
00021 #include <chrono>
00022 #include <limits>
00023 #include <fstream>
00024 #include <sstream>
00025 #include <list>
00026 #include <deque>
00027 #include <filesystem>
00028 #include <windows.h>
00029
00030 using std::cin;
00031 using std::cout;
00032 using std::string;
00033 using std::vector;
00034 using std::left;
00035 using std::right;
00036 using std::setw;
00037 using std::endl;
00038 using std::sort;
00039 using std::fixed;
00040 using std::setprecision;
```

```

00041 using std::ifstream;
00042 using std::ofstream;
00043 using std::getline;
00044 using std::cerr;
00045 using std::stringstream;
00046 using std::move;
00047 using std::ostream;
00048 using std::istream;
00049 using std::chrono::high_resolution_clock;
00050 using std::chrono::duration_cast;
00051 using std::chrono::duration;
00052 using std::chrono::milliseconds;
00053 using std::numeric_limits;
00054 using std::mt19937;
00055 using std::chrono::steady_clock;
00056 using std::invalid_argument;
00057 using std::out_of_range;
00058 using std::exception;
00059 using std::runtime_error;
00060 using std::streamsize;
00061 using std::to_string;
00062 using std::pair;
00063 using std::list;
00064 using std::deque;
00065 using std::stable_partition;
00066 using std::copy_if;
00067 using std::back_inserter;
00068 using std::make_move_iterator;
00069 namespace fs = std::filesystem;
00070
00071 // =====
00072 // Studentas klasė
00073 // Visi laukai yra private -- prieinama tik per metodus.
00074 // =====
00075
00076 class Zmogus {
00077 protected:
00078     // protected -- paveldėjusi klasė (Studentas) gali pasiekti tiesiogiai;
00079     // iš išorės -- tik per getter'ius/setter'ius.
00080     string vardas_;
00081     string pavarde_;
00082
00083 public:
00084     // ---- Konstruktoriai ----
00085     Zmogus() = default;
00086
00087     Zmogus(string v, string p)
00088         : vardas_(move(v)), pavarde_(move(p)) {}
00089
00090     // ---- Rule of Five baziniai metodai ----
00091     // = default -- naudojami kompiliatoriaus generuoti variantai,
00092     // nes laukai (string) patys tvarkosi su kopijavimu/perkėlimu.
00093     Zmogus(const Zmogus&) = default;
00094     Zmogus(Zmogus&&) noexcept = default;
00095     Zmogus& operator=(const Zmogus&) = default;
00096     Zmogus& operator=(Zmogus&&) noexcept = default;
00097
00098     // Virtualus destruktorius -- BŪTINA paveldėjimo bazei
00099     virtual ~Zmogus() = default;
00100
00101     // ---- Getteriai ----
00102     const string& vardas() const { return vardas_; }
00103     const string& pavarde() const { return pavarde_; }
00104
00105     // ---- Setteriai ----
00106     void setVardas(const string& v) { vardas_ = v; }
00107     void setPavarde(const string& p) { pavarde_ = p; }
00108
00109     // ---- Gryna virtuali funkcija ----
00110     // = 0 padaro Zmogų abstrakčia klase.
00111     // Kiekviena paveldėjusi klasė PRIVALO ją įgyvendinti.
00112     virtual void spausdinti(ostream& os) const = 0;
00113 };
00114
00115 class Studentas : public Zmogus {
00116 private:
00117     vector<int> paz_;
00118     int egz_ = 0;
00119     double gal_vid_ = 0.0;
00120     double gal_med_ = 0.0;
00121
00122 public:
00123     // =====
00124     // Konstruktoriai
00125     // =====
00126
00127     Studentas() = default;

```

```

00128
00129 // Pilnas konstruktorius -- perduodame v ir p į Zmogaus konstruktorių
00130 Studentas(string v, string p, vector<int> paz, int egz, int metodas)
00131 : Zmogus(move(v), move(p)), // <-- iškviečiamas bazinis konstruktorius
00132 paz_(move(paz)), egz_(egz)
00133 {
00134     apskaiciuoti(metodas);
00135 }
00136
00137 // =====
00138 // Rule of Five
00139 // Kiekviename privalu IŠKVIESTI bazinį (Zmogus) variantą,
00140 // kad vardas ir pavardė būtų teisingai nukopijuoti/perkelti.
00141 // =====
00142
00143 // Destruktorius -- override žymi, kad perdengiame bazinį virtualų
00144 ~Studentas() override = default;
00145
00146 // Kopijavimo konstruktorius
00147 Studentas(const Studentas& kitas)
00148 : Zmogus(kitas), // <-- bazinis copy ctor
00149 paz_(kitas.paz_),
00150 egz_(kitas.egz_),
00151 gal_vid_(kitas.gal_vid_),
00152 gal_med_(kitas.gal_med_)
00153 {}
00154
00155 // Kopijavimo priskyrimas
00156 Studentas& operator=(const Studentas& kitas) {
00157     if (this != &kitas) {
00158         Zmogus::operator=(kitas); // <-- bazinis copy assignment
00159         paz_ = kitas.paz_;
00160         egz_ = kitas.egz_;
00161         gal_vid_ = kitas.gal_vid_;
00162         gal_med_ = kitas.gal_med_;
00163     }
00164     return *this;
00165 }
00166
00167 // Perkėlimo konstruktorius
00168 Studentas(Studentas&& kitas) noexcept
00169 : Zmogus(move(kitas)), // <-- bazinis move ctor
00170 paz_(move(kitas.paz_)),
00171 egz_(kitas.egz_),
00172 gal_vid_(kitas.gal_vid_),
00173 gal_med_(kitas.gal_med_)
00174 {
00175     kitas.egz_ = 0;
00176     kitas.gal_vid_ = 0.0;
00177     kitas.gal_med_ = 0.0;
00178 }
00179
00180 // Perkėlimo priskyrimas
00181 Studentas& operator=(Studentas&& kitas) noexcept {
00182     if (this != &kitas) {
00183         Zmogus::operator=(move(kitas)); // <-- bazinis move assignment
00184         paz_ = move(kitas.paz_);
00185         egz_ = kitas.egz_;
00186         gal_vid_ = kitas.gal_vid_;
00187         gal_med_ = kitas.gal_med_;
00188         kitas.egz_ = 0;
00189         kitas.gal_vid_ = 0.0;
00190         kitas.gal_med_ = 0.0;
00191     }
00192     return *this;
00193 }
00194
00195 // =====
00196 // Virtualios funkcijos perdengimas
00197 // =====
00198
00199 // override -- kompiliatorius patikrina, kad funkcija TIKRAI
00200 // perdengia bazinę (jei užklysta klaida, neleis kompiliuoti).
00201 void spausdinti(ostream& os) const override {
00202     os << left << setw(15) << vardas_
00203         << setw(15) << pavarde_;
00204     os << "ND:";
00205     for (int p : paz_) os << " " << p;
00206     os << " Egz: " << egz_;
00207     if (gal_vid_ > 0.0)
00208         os << fixed << setprecision(2) << " Vid: " << gal_vid_;
00209     if (gal_med_ > 0.0)
00210         os << fixed << setprecision(2) << " Med: " << gal_med_;
00211 }
00212
00213
00214 // =====

```



```

00215 // Getteriai (papildomi prie paveldėtų iš Zmogus)
00216 // =====
00217
00218 const vector<int>& paz() const { return paz_; }
00219 int egz() const { return egz_; }
00220 double galVid() const { return gal_vid_; }
00221 double galMed() const { return gal_med_; }
00222
00223 // =====
00224 // Setteriai (papildomi)
00225 // =====
00226
00227 void setEgz(int e) { egz_ = e; }
00228 void addPazmys(int p) { paz_.push_back(p); }
00229
00230 void nustatytiEgzIsGalo() {
00231     if (!paz_.empty()) {
00232         egz_ = paz_.back();
00233         paz_.pop_back();
00234     }
00235 }
00236 // =====
00237 // Skaičiavimo metodai
00238 // =====
00239
00240 double skaiciuotiVidurki() const {
00241     if (paz_.empty()) return 0.0;
00242     double suma = 0.0;
00243     for (int p : paz_) suma += p;
00244     return suma / static_cast<double>(paz_.size());
00245 }
00246
00247 double skaiciuotiMediana() const {
00248     if (paz_.empty()) return 0.0;
00249     vector<int> tmp = paz_;
00250     sort(tmp.begin(), tmp.end());
00251     size_t n = tmp.size();
00252     if (n % 2 == 0)
00253         return (tmp[n / 2 - 1] + tmp[n / 2]) / 2.0;
00254     return tmp[n / 2];
00255 }
00256
00257 void apskaiciuoti(int metodas) {
00258     if (metodas == 1 || metodas == 3)
00259         gal_vid_ = skaiciuotiVidurki() * 0.4 + egz_ * 0.6;
00260     if (metodas == 2 || metodas == 3)
00261         gal_med_ = skaiciuotiMediana() * 0.4 + egz_ * 0.6;
00262 }
00263
00264 // =====
00265 // friend deklaracija -- operator» turi pasiekti private laukus
00266 // =====
00267 friend istream& operator»(istream& is, Studentas& st);
00268 };
00269 // =====
00270 // Įvesties/išvesties operatoriai (laisvos funkcijos, ne klasės nariai)
00271 // =====
00272
00273 // operator» -- priima Zmogų pagal nuorodą.
00274 // Kviečia virtualų spausdinti() -- automatiškai parinks teisingą
00275 // versiją (Studentas::spausdinti) per virtualų dispatch.
00276 //
00277 // Naudojimas:
00278 // Studentas st(...);
00279 // cout << st; // veikia
00280 // Zmogus& ref = st; // bazinė nuoroda
00281 // cout << ref; // veikia -- kviečia Studentas::spausdinti
00282 inline ostream& operator«(ostream& os, const Zmogus& z) {
00283     z.spausdinti(os);
00284     return os;
00285 }
00286
00287 // operator» -- tik Studentui (Zmogus per save savęs prirašyti negali).
00288 // Formatas: Vardas Pavarde ND1 ND2 ... NDn Egzaminas
00289 inline istream& operator«(istream& is, Studentas& st) {
00290     string eilute;
00291     if (!getline(is, eilute)) return is;
00292     if (eilute.empty()) return is;
00293
00294     stringstream ss(eilute);
00295     string v, p;
00296     if (!(ss >> v >> p)) return is;
00297
00298     st.vardas_ = v; // pasiekiama nes operator» yra friend
00299     st.pavarde_ = p;
00300     st.paz_.clear();
00301     st.egz_ = 0;

```

```

00302     st.gal_vid_ = 0.0;
00303     st.gal_med_ = 0.0;
00304
00305     int n;
00306     while (ss > n) st.paz_.push_back(n);
00307
00308     if (!st.paz_.empty()) {
00309         st.egz_ = st.paz_.back();
00310         st.paz_.pop_back();
00311     }
00312     return is;
00313 }
00314
00315 // =====
00316 // Laisvų funkcijų prototipai
00317 // =====
00318
00319 int gautiSkaiciu(string info, int min, int max);
00320 string genVarda();
00321 string genPavarde(string vardas);
00322 void genPazymius(vector<int>& paz, int& egz);
00323
00324 void genFaila(const string& failas, int kiek);
00325
00326 void skaitytiVector(const string& failas, vector<Studentas>& grupe, int metodas);
00327 void skaitytiList(const string& failas, list<Studentas>& grupe, int metodas);
00328 void skaitytiDeque(const string& failas, deque<Studentas>& grupe, int metodas);
00329
00330 void skirstytiVector(const vector<Studentas>& visi, vector<Studentas>& kieti, vector<Studentas>&
tinginiai);
00331 void skirstytiList(const list<Studentas>& visi, list<Studentas>& kieti, list<Studentas>&
tinginiai);
00332 void skirstytiDeque(const deque<Studentas>& visi, deque<Studentas>& kieti, deque<Studentas>&
tinginiai);
00333
00334 void skaitytiIsFailo(const string& failas, vector<Studentas>& grupe, int metodas);
00335 void spausdintiRezultatus(const vector<Studentas>& grupe, int rodyti, const string& failas);
00336 void splitStudents(const vector<Studentas>& visi, vector<Studentas>& kieti,
vector<Studentas>& tinginiai, int metodas);
00337 void vykdytiVector();
00338
00339 void test1();
00340 void test2(const string& failas, int metodas);

```

6.5 Uzd2.cpp Failo Nuoroda

```
#include "struktura.h"
```

Funkcijos

- int [main](#) ()

6.5.1 Funkcijos Dokumentacija

6.5.1.1 main()

```
int main ()
```

6.6 vektorius.cpp Failo Nuoroda

```
#include "struktura.h"
```

Funkcijos

- void [skaitytiIsFailo](#) (const string &failas, vector< [Studentas](#) > &grupe, int metodas)
- void [spausdintiRezultatus](#) (const vector< [Studentas](#) > &grupe, int rodyti, const string &failas)
- void [splitStudents](#) (const vector< [Studentas](#) > &visi, vector< [Studentas](#) > &kieti, vector< [Studentas](#) > &tinginiai, int metodas)
- void [vykdytiVector](#) ()

Kintamieji

- static const string `DATA_DIR` = "Data/"

6.6.1 Funkcijos Dokumentacija

6.6.1.1 skaitytiIsFailo()

```
void skaitytiIsFailo (  
    const string & failas,  
    vector< Studentas > & grupe,  
    int metodas)
```

6.6.1.2 spausdintiRezultatus()

```
void spausdintiRezultatus (  
    const vector< Studentas > & grupe,  
    int rodyti,  
    const string & failas)
```

6.6.1.3 splitStudents()

```
void splitStudents (  
    const vector< Studentas > & visi,  
    vector< Studentas > & kieti,  
    vector< Studentas > & tinginiai,  
    int metodas)
```

6.6.1.4 vykdytiVector()

```
void vykdytiVector ()
```

6.6.2 Kintamojo Dokumentacija

6.6.2.1 DATA_DIR

```
const string DATA_DIR = "Data/" [static]
```


Rodyklė

- ~Studentas
 - Studentas, [14](#)
- ~Zmogus
 - Zmogus, [17](#)
- addPazymys
 - Studentas, [14](#)
- apskaiciuoti
 - Studentas, [14](#)
- DATA_DIR
 - vektorius.cpp, [29](#)
- egz
 - Studentas, [15](#)
- egz_
 - Studentas, [16](#)
- funkcijos.cpp, [19](#)
 - gautiSkaiciu, [20](#)
 - genFaila, [20](#)
 - genPavarde, [20](#)
 - genPazymius, [20](#)
 - genVarda, [20](#)
 - mt, [20](#)
 - NOMINMAX, [19](#)
 - skaitytiDeque, [20](#)
 - skaitytiList, [20](#)
 - skaitytiVector, [20](#)
 - skirstytiDeque, [20](#)
 - skirstytiList, [21](#)
 - skirstytiVector, [21](#)
 - WIN32_LEAN_AND_MEAN, [19](#)
- gal_med_
 - Studentas, [16](#)
- gal_vid_
 - Studentas, [16](#)
- galMed
 - Studentas, [15](#)
- galVid
 - Studentas, [15](#)
- gautiSkaiciu
 - funkcijos.cpp, [20](#)
 - struktura.h, [22](#)
- genFaila
 - funkcijos.cpp, [20](#)
 - struktura.h, [22](#)
- genPavarde
 - funkcijos.cpp, [20](#)
- struktura.h, [22](#)
- genPazymius
 - funkcijos.cpp, [20](#)
 - struktura.h, [22](#)
- genVarda
 - funkcijos.cpp, [20](#)
 - struktura.h, [22](#)
- main
 - Uzd2.cpp, [28](#)
- mt
 - funkcijos.cpp, [20](#)
- NOMINMAX
 - funkcijos.cpp, [19](#)
 - struktura.h, [22](#)
- nustatytiEgzIsGalo
 - Studentas, [15](#)
- operator<<
 - struktura.h, [22](#)
- operator>>
 - struktura.h, [23](#)
 - Studentas, [15](#)
- operator=
 - Studentas, [15](#)
 - Zmogus, [17](#)
- pavarde
 - Zmogus, [17](#)
- pavarde_
 - Zmogus, [17](#)
- paz
 - Studentas, [15](#)
- paz_
 - Studentas, [16](#)
- Pirmoji užduotis — v1.0 pre-release, [1](#)
- README.md, [21](#)
- setEgz
 - Studentas, [15](#)
- setPavarde
 - Zmogus, [17](#)
- setVardas
 - Zmogus, [17](#)
- skaiciuotiMediana
 - Studentas, [15](#)
- skaiciuotiVidurki
 - Studentas, [15](#)

skaitytiDeque
 funkcijos.cpp, 20
 struktura.h, 23
 skaitytiFailo
 struktura.h, 23
 vektorius.cpp, 29
 skaitytiList
 funkcijos.cpp, 20
 struktura.h, 23
 skaitytiVector
 funkcijos.cpp, 20
 struktura.h, 23
 skirstytiDeque
 funkcijos.cpp, 20
 struktura.h, 23
 skirstytiList
 funkcijos.cpp, 21
 struktura.h, 23
 skirstytiVector
 funkcijos.cpp, 21
 struktura.h, 23
 spausdinti
 Studentas, 15
 Zmogus, 17
 spausdintiRezultatus
 struktura.h, 23
 vektorius.cpp, 29
 splitStudents
 struktura.h, 24
 vektorius.cpp, 29
 struktura.h, 21
 gautiSkaiciu, 22
 genFaila, 22
 genPavarde, 22
 genPazymius, 22
 genVarda, 22
 NOMINMAX, 22
 operator<<, 22
 operator>>, 23
 skaitytiDeque, 23
 skaitytiFailo, 23
 skaitytiList, 23
 skaitytiVector, 23
 skirstytiDeque, 23
 skirstytiList, 23
 skirstytiVector, 23
 spausdintiRezultatus, 23
 splitStudents, 24
 test1, 24
 test2, 24
 vykdytiVector, 24
 WIN32_LEAN_AND_MEAN, 22
 Studentas, 13
 ~Studentas, 14
 addPazymys, 14
 apskaiciuoti, 14
 egz, 15
 egz_, 16
 gal_med_, 16
 gal_vid_, 16
 galMed, 15
 galVid, 15
 nustatytiEgzIsGalo, 15
 operator>>, 15
 operator=, 15
 paz, 15
 paz_, 16
 setEgz, 15
 skaiciuotiMediana, 15
 skaiciuotiVidurki, 15
 spausdinti, 15
 Studentas, 14
 test1
 struktura.h, 24
 test2
 struktura.h, 24
 Uzd2.cpp, 28
 main, 28
 vardas
 Zmogus, 17
 vardas_
 Zmogus, 17
 vektorius.cpp, 28
 DATA_DIR, 29
 skaitytiFailo, 29
 spausdintiRezultatus, 29
 splitStudents, 29
 vykdytiVector, 29
 vykdytiVector
 struktura.h, 24
 vektorius.cpp, 29
 WIN32_LEAN_AND_MEAN
 funkcijos.cpp, 19
 struktura.h, 22
 Zmogus, 16
 ~Zmogus, 17
 operator=, 17
 pavarde, 17
 pavarde_, 17
 setPavarde, 17
 setVardas, 17
 spausdinti, 17
 vardas, 17
 vardas_, 17
 Zmogus, 16, 17